College AI Assistant: A Multi-Signal, Domain-Constrained Conversational System

Joseph Jonathan Fernandes

Applications of AI Project Report

I. Literature Survey

Conversational AI systems are commonly organized as (i) an NLU layer that maps user language to structured intent/state and (ii) a response layer that produces natural language outputs. The selected works below are consolidated and rewritten using the technically strongest explanations from the available draft surveys, while preserving a logical progression from traditional sequence models, to transformer-based representation learning, to dialogue alignment, and finally to intent evaluation with explicit out-of-scope (OOS) handling.

A. Traditional Methods → Transformers → Dialogue Systems → Intent Evaluation

1) Hussain et al. (2019) — Conversational agent taxonomy and design techniques [1]: **Problem:** The chatbot design space lacks a consistent taxonomy, making it difficult to compare systems and reason about controllability versus coverage.

Method: Systematic survey that classifies agents as rule-based, retrieval-based, or generative and summarizes common implementation patterns.

Contribution: Establishes an organizing framework that motivates domain scoping and hybrid architectures for constrained deployments.

Limitation: Characterization of generative systems largely predates large-scale transformer alignment methods, underrepresenting RLHF-era safety and preference optimization.

2) Caldarini et al. (2022) — Recent chatbot advances and evaluation practices [2]: **Problem:** Rapid growth of chatbot research has produced fragmented evaluation practices and inconsistent reporting across systems and domains.

Method: Systematic literature survey of modern chatbot architectures, datasets, and evaluation methods (2019–2021).

Contribution: Provides consolidated trends (architecture families, application domains, evaluation protocols) and highlights the continued reliance on human evaluation.

Limitation: As a survey, it does not provide deployable algorithms and cannot resolve limitations of the underlying benchmarks.

3) Liu and Lane (2016) — Joint intent classification and slot filling [3]: **Problem:** Pipeline NLU architectures propagate errors between intent detection and slot filling.

Method: Attention-based bidirectional RNN trained jointly for utterance-level intent classification and token-level slot tagging.

Contribution: Demonstrates that shared representations improve joint performance compared to independent pipeline components.

Limitation: Sequential recurrence reduces parallelism and long-context modeling; the formulation is utterance-centric rather than multi-turn.

4) Hakkani-Tur et al. (2016) — Multi-domain joint semantic frame parsing [4]: **Problem:** Real assistants must support multiple domains while maintaining consistent semantic parsing quality.

Method: BiRNN-LSTM model trained to jointly predict domain/intent/slots, enabling shared representations across domains.

Contribution: Shows that multi-domain joint training can improve low-resource domain performance through transfer.

Limitation: Fixed ontologies and domain boundaries remain brittle; scaling to many domains increases ambiguity and motivates explicit OOS handling.

5) Vaswani et al. (2017) — Transformer self-attention [5]: **Problem:** RNN-based sequence-to-sequence models are inherently sequential and struggle with long-range dependencies.

Method: Encoder–decoder transformer using multi-head scaled dot-product self-attention and positional encodings.

Contribution: Replaces recurrence with attention, enabling parallel training and establishing the foundation for modern pre-trained models.

Limitation: Self-attention has quadratic $O(n^2)$ cost in sequence length, limiting practical context windows without further architectural changes.

6) Devlin et al. (2019) — BERT bidirectional pre-training for NLU [6]: **Problem:** Unidirectional pre-training yields weaker contextual embeddings for classification and extraction tasks.

Method: Bidirectional transformer encoder pre-trained with masked language modeling (and next sentence prediction), then fine-tuned.

Contribution: Produces transfer-ready contextual representations that substantially improve intent classification and slot tagging without manual feature engineering.

Limitation: Encoder-only architecture is not a response generator and can be overconfident without explicit calibration and OOS-aware routing.

7) Liu et al. (2019) — RoBERTa and robust pre-training [7]: **Problem:** Downstream performance can be limited by suboptimal pre-training choices rather than model architecture.

Method: Refines BERT-style training by removing next sentence prediction, training longer on more data with larger batches, and using dynamic masking.

Contribution: Demonstrates that scaling data/compute and simplifying objectives yield consistent improvements, strengthening transformer encoders as NLU backbones.

Limitation: Improved representations do not eliminate the need for domain scoping, OOS detection, and uncertainty-aware decision thresholds.

8) Radford et al. (2018, 2019) — GPT-style generative pre-training [8], [9]: **Problem:** Task-specific modeling does not scale across diverse language tasks, limiting reuse across conversational settings.

Method: Autoregressive transformer decoder pre-training followed by task adaptation (fine-tuning or prompt conditioning).

Contribution: Establishes generative pre-training as a practical response-layer foundation and motivates prompt-based adaptation.

Limitation: Unconstrained generation can hallucinate domain facts, motivating guardrails, grounding, and scope control in deployed assistants.

9) Brown et al. (2020) — Few-shot learning with large language models [10]: **Problem:** Fine-tuning for each new task or domain is expensive and can fail to generalize under distribution shift.

Method: Scaling autoregressive transformers and evaluating in-context few-shot learning via prompt demonstrations.

Contribution: Shows prompting as an adaptation interface, enabling rapid task conditioning without gradient updates.

Limitation: High inference cost and limited grounding yield fluent but incorrect outputs, especially in constrained domains.

10) Khatri et al. (2018) — Alexa Prize: open-domain system lessons [11]: **Problem:** Academic metrics underrepresent the system-integration challenges of sustained open-domain dialogue with real users.

Method: Competition-driven deployment of socialbots combining topic tracking, retrieval, generation, and multi-skill orchestration.

Contribution: Highlights that robust user experience depends on hybrid architectures, graceful failure handling, and long-run evaluation.

Limitation: Results are difficult to reproduce outside the Alexa ecosystem, and user ratings are noisy and confounded by context.

11) Roller et al. (2021) — Recipes for building open-domain chatbots [12]: **Problem:** Purely generative dialogue models can be inconsistent and hard to control across topics and turns.

Method: Empirical "recipes" blending retrieval augmentation, generation, and evaluation discipline.

Contribution: Motivates multi-signal orchestration that combines deterministic control with generation, emphasizing human evaluation.

Limitation: Many practices assume large-scale data and annotation budgets that are not available in small projects.

12) Ouyang et al. (2022) — RLHF for instruction following (InstructGPT) [13]: **Problem:** Next-token pre-training optimizes likelihood rather than helpfulness, honesty, and harmlessness.

Method: Supervised fine-tuning on demonstrations, reward modeling from human preferences, and policy optimization (RLHF).

Contribution: Improves instruction adherence and reduces undesirable outputs, forming a widely adopted alignment template.

Limitation: Human feedback is costly and can encode annotator bias; reward models can be exploited by optimization artifacts.

13) Thoppilan et al. (2022) — Dialogue-optimized language models (LaMDA) [14]: **Problem:** General-purpose PLMs do not directly optimize dialogue-specific qualities such as safety and groundedness.

Method: Dialogue-focused training and tuning with objectives emphasizing quality, safety, and groundedness.

Contribution: Demonstrates that dialogue-specialized optimization improves human-rated conversational behavior.

Limitation: Resource requirements and residual factual errors motivate additional guardrails (scope control, grounding, and monitoring).

14) OpenAI (2022) — ChatGPT as a dialogue-aligned assistant [15]: **Problem:** Large language models require alignment to behave as usable assistants in interactive dialogue.

Method: Dialogue-focused fine-tuning and preference optimization in the RLHF family.

Contribution: Demonstrates that assistant behavior can be shaped toward instruction adherence and improved conversational usability.

Limitation: Alignment improves helpfulness but does not guarantee factual grounding, particularly for restricted domains.

15) Gao et al. (2019) — Neural conversational AI: system-level overview [16]: **Problem:** Model-centric summaries understate the system-level challenges of building conversational AI across paradigms and tasks.

Method: Tutorial-style synthesis of neural approaches spanning QA, task-oriented dialogue, and social chatbots.

Contribution: Provides an organizing perspective for how NLU, state tracking, response generation, and evaluation interact in end-to-end systems.

Limitation: As a survey/tutorial, it does not prescribe a single deployable architecture and abstracts away domain-specific constraints.

B. Intent and Dialogue Understanding Components

1) Larson et al. (2019) — CLINC150 with explicit OOS evaluation [17]: **Problem:** Many intent benchmarks omit OOS as a primary requirement, despite its centrality in deployed assistants.

Method: Intent dataset spanning many intents/domains and including an explicit OOS label for rejection evaluation.

Contribution: Operationalizes OOS detection and motivates calibrated confidence thresholds for deployment.

Limitation: OOS distributions vary by application and organization, requiring domain-specific validation beyond a single benchmark.

2) Casanueva et al. (2020) — Banking77 and efficient intent detection [18]: **Problem:** Large transformer classifiers can be too slow or costly for interactive intent routing.

Method: Dual sentence encoders with similarity-based classification, evaluated on the Banking77 dataset.

Contribution: Motivates embedding-based intent matching as an efficient alternative or complement to heavier encoders.

Limitation: Domain specificity limits transfer, and single-turn intent evaluation omits multi-turn state tracking.

3) Chen et al. (2019) — BERT for joint intent classification and slot filling [19]: **Problem:** Traditional joint NLU models require task-specific feature engineering and struggle to exploit deep contextual cues.

Method: Fine-tunes BERT for intent (via [CLS]) and slots (token-level tagging, commonly paired with a CRF).

Contribution: Establishes a strong joint NLU baseline with substantial gains over BiLSTM-based models.

Limitation: Inference cost can be high for latency-sensitive systems, and closed-set benchmarks may overestimate real-world robustness.

4) Xia et al. (2018) — Capsule networks for joint slot and intent [20]: **Problem:** Flat sequence encoders may fail to model hierarchical relationships between slot spans and intent labels.

Method: Capsule networks with dynamic routing to represent part–whole relationships between tokens, slots, and intents.

Contribution: Introduces a structured inductive bias that can improve robustness on ambiguous utterances.

Limitation: Routing adds complexity and compute; benefits can be sensitive to dataset characteristics.

5) Zhang et al. (2020) — Few-shot intent detection [21]: **Problem:** New intents emerge frequently, but labeled data for each new intent is scarce.

Method: Contrastive pre-training to learn intent-discriminative representations, followed by few-shot fine-tuning.

Contribution: Improves intent generalization under limited supervision, supporting modular onboarding of new intents.

Limitation: Performance is sensitive to negative sampling and dataset construction; extreme low-shot settings remain challenging.

6) Kim et al. (2019) — Dialogue state tracking as reading comprehension [22]: **Problem:** Dialogue state tracking must extract slot values over multi-turn history, including values not seen during training.

Method: Reformulates DST as reading comprehension over dialogue history, extracting slot values by querying the context.

Contribution: Provides an ontology-light approach that generalizes to open-vocabulary values better than closed-set classification.

Limitation: Per-slot querying can increase inference cost, and implicit/unspoken values remain difficult to recover.

7) Huang et al. (2020) — TRADE: transferable dialogue state generator [23]: **Problem:** Multi-domain DST requires tracking many slots and unseen values without domain-specific parameterization.

Method: State generator with a copy mechanism that generates slot values token-by-token from dialogue context.

Contribution: Enables cross-domain transfer and handles unseen slot values by copying from context.

Limitation: Autoregressive generation increases latency and can degrade on long contexts or long-tail value distributions.

8) Maroengsit et al. (2019) — Evaluation methods for chatbots [24]: **Problem:** Chatbot quality spans multiple dimensions that are poorly captured by single automatic metrics.

Method: Survey of automatic metrics and human evaluation protocols across chatbot types.

Contribution: Encourages multi-metric reporting and careful interpretation of automated scores.

Limitation: Many automatic metrics correlate weakly with human judgments in open-domain dialogue.

C. Dataset Artifacts, Robustness, and Deployment Risk

1) Gururangan et al. (2018) — Annotation artifacts in NLU benchmarks [25]: **Problem:** Benchmark performance can be inflated when datasets contain spurious lexical artifacts that correlate with labels.

Method: Artifact analysis showing that models can succeed by exploiting shallow correlations.

Contribution: Motivates skepticism toward single-dataset accuracy claims and supports robustness-oriented evaluation.

Limitation: Artifact detection identifies risk but does not automatically produce domain-specific mitigations.

2) Geva et al. (2019) — Annotator bias and shortcut learning [26]: **Problem:** Models may learn annotator-specific patterns rather than the intended semantic task.

Method: Empirical analysis isolating annotator signals and shortcut features in NLU datasets.

Contribution: Highlights measurement error in reported performance and motivates diverse, well-controlled annotation protocols.

Limitation: Bias diagnosis requires careful experimental design and can be difficult to integrate into small-project evaluations.

3) Swayamdipta et al. (2020) — Dataset cartography via training dynamics [27]: **Problem:** Aggregate accuracy obscures which examples are easy, ambiguous, or systematically hard for models.

Method: Uses training dynamics (confidence and variability across epochs) to map instances into difficulty regimes.

Contribution: Provides a practical lens for diagnosing dataset difficulty and targeting evaluation/augmentation.

Limitation: Requires instrumented training runs and is sensitive to model/training configuration.

4) Belinkov and Bisk (2018) — Noise sensitivity in neural models [28]: **Problem:** Neural sequence models degrade sharply under realistic input noise (typos, character-level perturbations).

Method: Evaluates robustness under synthetic and natural noise and quantifies performance degradation.

Contribution: Motivates explicit preprocessing and robustness testing for deployed NLU components.

Limitation: Findings generalize broadly but do not prescribe a single mitigation strategy suitable for all domains.

5) Sun et al. (2020) — Adversarial training for code-mixed/noisy inputs [29]: **Problem:** Language understanding in multilingual and code-mixed user inputs is brittle under distribution shift.

Method: Adversarial training strategies to improve robustness to perturbations and mixing patterns.

Contribution: Supports robustness-oriented evaluation and defensive training for real-world conversational inputs.

Limitation: Robustness techniques can be data- and compute-intensive relative to small-project constraints.

Collectively, the surveyed literature motivates a hybrid architecture in which deterministic control (intent routing, scope gating, and monitoring) complements generative response modeling. The following section formalizes this design for a college-domain assistant and specifies the pipeline and decision points that guide the repository implementation.

II. Methodology (Design)

The system is designed as a domain-constrained conversational assistant that combines a structured NLU/control layer with a controlled response generation layer. This hybrid approach follows system-level findings that robust assistants require orchestration, context handling, uncertainty management, and evaluation discipline in addition to fluent text generation [2], [11], [16], [24], [30].

A. System Overview

The design targets a college-information domain and is organized as a multi-signal pipeline:

- **Control and understanding:** preprocessing, intent inference, uncertainty estimation, scope (in-scope/OOS) decisions, and conversation context.
- **Response layer:** knowledge-grounded response planning followed by controlled natural language generation.
- **Monitoring and evaluation:** telemetry signals (intent confidence, latency, error modes) and test-driven regression checks to track reliability.

This decomposition reflects common taxonomies for conversational agents and supports predictable behavior under ambiguous or out-of-scope inputs [1], [17], [18].

B. Architecture Diagram (Figure 1)

Figure 1 summarizes the repository-grounded architecture. The design separates local control decisions (preprocessing, intent/emotion/scope inference, context) from external LLM response generation. This separation aligns with deployment guidance emphasizing modular guardrails and explicit control points for conversational systems [11]–[13].

C. Data Flow Pipeline

The end-to-end pipeline is defined as a deterministic sequence of transformations and decisions:

Input → Preprocessing → Representation → Intent/Scope Decision → Response Planning → Generation → Output

More explicitly:

1) **Input:** user utterance u_t.
2) **Preprocessing:** normalization to reduce sensitivity to character-level noise that can degrade NLU performance [28], [29].
3) **Representation:** a lexical vector $v(u_t)$ (sparse) and a semantic embedding $e(u_t)$ (transformer encoder) [6], [7].
4) **Intent inference:** compute intent scores and aggregate them into a calibrated posterior $p(y \mid u_t)$ over intents.
5) **Uncertainty and scope:** compute confidence $c = \max_y p(y \mid u_t)$ and apply thresholds to accept, clarify, or reject (OOS) [17], [18].
6) **Response planning:** condition on (u_t, m_t) and retrieved facts k_t to form a response plan that constrains generation.
7) **Generation and output:** produce the final natural language response with conservative behavior under uncertainty [9], [13], [14].

D. Model Design (intent classification + response generation)

1) Intent classification: Intent classification is modeled as multi-class prediction over a closed intent set. Given tokenized input $x = [x_1, \ldots, x_n]$, a transformer encoder produces contextual states $H = [h_1, \ldots, h_n]$ [5], [6]. A pooled vector z is mapped to a posterior over intents:

$$p(y \mid x) = \mathrm{softmax}(Wz + b), \quad c = \max_y p(y \mid x).$$

For efficiency, inference may be complemented with similarity-based retrieval in embedding space, where each intent i has a prototype text t_i and score

$$s_i = \cos\big(e(u_t), e(t_i)\big), \quad \hat{y} = \arg\max_i s_i.$$

The final decision uses uncertainty-aware thresholds (e.g., reject if $c < \tau$ or $\max_i s_i < \gamma$), reflecting the need for calibrated OOS behavior [17], [18], [21].

TABLE I
COMPARISON TABLE (SELECTED WORKS)

Paper	Method	Strength	Limitation
[5]	Transformer	Parallelizable; long-range modeling	$O(n^2)$ attention cost
[6]	BERT pre-training	Strong NLU transfer	Encoder-only; calibration/OOS needed
[9]	GPT-style pre-training	Strong generative response layer	Hallucination/grounding risk
[13]	RLHF alignment	Better instruction following	Costly and bias-sensitive
[14]	Dialogue LM tuning	Improves safety/groundedness focus	Resource-intensive; residual errors
[19]	Joint NLU with BERT	Strong intent/slot baseline	Compute/latency constraints
[17]	OOS benchmark	Tests OOS rejection explicitly	OOS varies by domain
[18]	Dual encoder	Efficient inference	Domain-specific; single-turn
[22]	DST as RC	Open-vocabulary slot extraction	Per-slot cost; implicit values
[28]	Noise robustness eval	Quantifies brittleness under noise	Mitigation not guaranteed

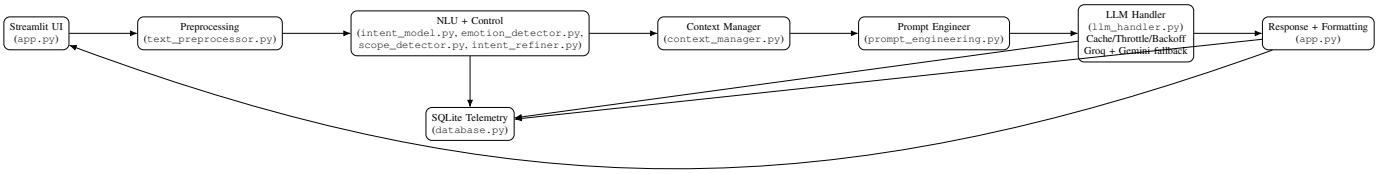

Fig. 1. Architecture of the college-domain assistant. The NLU/control layer runs locally, while response generation is delegated to an external LLM endpoint with caching, throttling, and fallback.

2) Response generation: Response generation is treated as conditional text generation using an autoregressive decoder-style language model (LM). Given a structured response plan and grounding context, generation models the probability of an output sequence $r = [r_1, \ldots, r_T]$ as

$$p(r \mid u_t, m_t, k_t) = \prod_{t=1}^{T} p(r_t \mid r_{<t}, u_t, m_t, k_t).$$

Decoder-only pre-training establishes the base generative capability [8]–[10]. Alignment and dialogue specialization approaches improve instruction adherence and conversational quality [13]–[15], but they do not guarantee domain factuality; therefore, grounding and scope control remain first-class design requirements [11], [12].

E. Algorithms / Models Used (RNN vs Transformer comparison)

Traditional joint NLU systems often rely on recurrent encoders (BiRNN/BiLSTM) trained jointly for intent and slots [3], [4]. Recurrence provides a strong inductive bias for local sequential patterns but is inherently sequential and can limit throughput.

Transformer encoders replace recurrence with self-attention, enabling parallel training and stronger long-range context modeling [5]. Pre-trained transformer encoders (e.g., BERT/RoBERTa) provide transferable representations that improve intent classification and joint intent/slot baselines [6], [7], [19].

For response generation, decoder-style transformers support fluent multi-turn responses but require alignment and system constraints to mitigate hallucination risk [9], [13]. End-to-end trainable dialogue systems have also been explored, including task-oriented dialogue networks and memory-based approaches [31], [32]; however, these formulations provide fewer explicit control points for domain scoping and safety policies.

F. Design Justification (why chosen)

The design adopts a modular, hybrid architecture rather than a fully end-to-end conversational model because modular decomposition improves controllability and allows explicit enforcement of domain scope, uncertainty handling, and evaluation checkpoints [1], [11], [12], [16].

Uncertainty-aware intent routing is emphasized because deployed assistants must handle ambiguous and out-of-scope inputs; explicit OOS gating is supported by intent benchmarks designed to measure rejection performance [17], [18]. A hybrid intent stack (lexical evidence + transformer-derived semantics) supports a practical trade-off among interpretability, latency, and paraphrase robustness [6], [7].

Robustness and measurement risks motivate conservative preprocessing and careful interpretation of automated evaluation results. Neural NLU brittleness under noise and dataset artifacts can inflate reported performance without improving real-world reliability [25]–[29]. The methodology therefore prioritizes scope control and grounded generation over unconstrained fluency, consistent with survey evidence that reliable chatbot performance depends on system integration and evaluation discipline [2], [24]. The following section maps this design to concrete modules and execution pathways in the repository.

III. IMPLEMENTATION

This section operationalizes the design using Python modules in the repository. The implementation emphasizes reproducible preprocessing, explicit scope and confidence controls, and a monitored LLM response layer.

A. Development Environment (languages, tools, libraries)

Language and runtime. The system is implemented in Python and executed as either a Streamlit web application (interactive UI) or a command-line runner.

Core libraries.

- **UI:** Streamlit for the chat interface and session state management.
- **NLU models:** PyTorch as the deep learning backend, Hugging Face Transformers for transformer pipelines, and Sentence-Transformers for embedding-based semantic similarity.
- **Classical ML:** scikit-learn for TF–IDF vectorization and logistic regression.
- **Data and utilities:** NumPy/Pandas for numeric and tabular handling; Requests for HTTP.
- **LLM providers:** Groq client/HTTP for the primary model endpoint and `google-genai` for Gemini fallback; `python-dotenv` for environment-variable configuration.
- **Persistence:** SQLite (via the Python standard library) for interaction logging and lightweight analytics.

Testing and quality tools. The development toolchain includes PyTest (with coverage, parallelism, and timeouts), Hypothesis for property-based tests, Locust for load/performance testing, and formatting/linting tools (Black, Flake8, isort).

B. Dataset Description (source, preprocessing steps)

Two local JSON artifacts serve as the primary data sources:

- **Intent dataset** (`data/intents.json`): an intent schema containing tags, paraphrased training patterns, and optional keyword lists. This file is used to train lightweight intent components at application startup.
- **Domain knowledge base** (`data/college_data.json`): structured college information (fees, exams, placements, hostel, admissions, etc.) used for retrieval of relevant facts for response grounding.

Preprocessing. User input and intent patterns are normalized using a dedicated preprocessing stage (`text_preprocessor.py`). Key transformations include typo normalization and abbreviation expansion, Hinglish/code-switch mapping for common Hindi–English tokens, and regex-based cleanup for punctuation artifacts and whitespace normalization. This preprocessing targets practical robustness issues where minor orthographic noise can degrade NLU signals [28], [29].

C. Model Implementation (training, fine-tuning)

The implementation uses pre-trained transformer models without project-specific fine-tuning of large encoders/decoders. Instead, fast-to-train components are fit on local intent patterns, while response generation relies on instruction-following LLM inference.

1) Intent components (startup training):

- **Semantic classifier:** a Sentence-Transformers encoder (default: `all-MiniLM-L6-v2`) embeds each preprocessed intent pattern; inference selects the intent with maximal cosine similarity to the user embedding.
- **TF–IDF classifier:** a scikit-learn TF–IDF vectorizer (unigrams/bigrams) and logistic regression classifier are trained on the same preprocessed patterns.
- **Ensembling and calibration:** the final intent prediction uses weighted aggregation with explicit calibration heuristics (agreement boosts, scaling factors, confidence floors) to improve stability under paraphrases and short/noisy inputs.

This hybrid intent implementation reflects the practical combination of transformer representations [5]–[7] and efficient similarity-based intent matching [18].

2) Emotion signal (pre-trained inference): A cached transformer pipeline performs sentiment analysis and maps sentiment to coarse interaction emotions (e.g., neutral, stressed). Keyword-based filters reduce false positives on short information-seeking questions.

3) LLM response generation (inference, no fine-tuning): Response generation is performed via an autoregressive LLM endpoint (Groq, with Gemini fallback), consistent with decoder-only pre-training and instruction-following alignment approaches [9], [13], [15]. The implementation does not train a dialogue model from scratch; it conditions inference using structured prompts and retrieved domain facts.

D. System Integration (frontend/backend if any)

Frontend. The Streamlit application orchestrates the end-to-end turn loop (message capture, inference calls, UI rendering) and maintains session identifiers for logging.

Backend orchestration. The orchestration layer instantiates and caches major components (intent, scope, emotion, prompt builder, LLM handler, and context manager) to amortize model-loading cost.

Persistence and analytics. A SQLite-backed logging layer records timestamped interactions with intent/confidence, emotion, scope decisions, response time, and LLM source. A small connection pool reduces per-turn database overhead.

E. Key Functional Modules

1) Intent classification: Intent classification is implemented as an ensemble: semantic similarity over transformer sentence embeddings; TF–IDF + logistic regression for lexical pattern capture; and a context-aware intent refiner that adjusts ambiguous predictions using recent-turn intent continuity.

Pipeline (pseudo-code style).

1) $u_t \leftarrow \text{user_message}$
2) $u'_t \leftarrow \text{preprocess}(u_t)$
3) $(y_{\text{sem}}, c_{\text{sem}}) \leftarrow \text{semantic_predict}(u'_t)$
4) $(y_{\text{lex}}, c_{\text{lex}}) \leftarrow \text{tfidf_predict}(u'_t)$
5) $(y, c) \leftarrow \text{weighted_ensemble}(y_{\text{sem}}, c_{\text{sem}}, y_{\text{lex}}, c_{\text{lex}})$
6) $(y, c) \leftarrow \text{refine_with_history}(y, c, \text{history}, \text{emotion})$

To support deployment realism, the intent decision is followed by explicit confidence thresholding and OOS handling aligned with benchmarks that emphasize rejection performance [17].

2) Entity/slot extraction: The implementation includes lightweight entity extraction in the context manager to support multi-turn continuity rather than full named-entity recognition. Extracted entities include user name patterns and intent-specific fields (e.g., department/program hints, exam type, fee-related subtopics). Pronoun resolution uses the most recent tracked entities to reduce context loss in follow-up turns.

3) Response generation: Response generation is implemented as a controlled LLM call:

- **Prompt construction:** a prompt-engineering module injects detected intent, confidence, scope status, emotion, conversation history, and retrieved knowledge snippets.
- **Knowledge grounding:** relevant sections of the college JSON are selected as context, with fallback to generic templates when a requested field is absent.
- **Operational safeguards:** caching for repeated queries, throttling between calls, concurrency limiting, exponential backoff with jitter, and API-key rotation; if the primary provider fails, the system falls back to Gemini.

These controls are implemented to reduce hallucination risk and stabilize user experience under rate limits and variable latency [11], [12].

F. Challenges Faced and Solutions

Noisy and code-mixed inputs. Short, misspelled, or Hinglish queries can degrade lexical and embedding signals. A dedicated normalization layer and dual-model ensemble mitigate brittleness [28], [29].

Out-of-scope (OOS) behavior. Closed-set intent classifiers are prone to overconfident misrouting for unsupported requests. The implementation includes explicit scope detection and confidence-based gating, consistent with OOS-oriented intent benchmarks [17], [18].

Latency and rate limiting for LLM calls. External LLM endpoints introduce variable response times and occasional rate limits. The implementation uses caching, throttling, concurrency limits, and retry/backoff policies, with provider fallback to maintain responsiveness.

Multi-turn coherence. Follow-up questions often omit explicit context (e.g., "What about that?"). A bounded conversation history and lightweight entity tracking reduce context loss without requiring full dialogue-state tracking models.

This implementation provides the concrete basis for the empirical results reported in the following section.

IV. Results

This section reports quantitative outcomes from (i) a component-level intent-classification evaluation using the repository intent patterns and (ii) an end-to-end automated test run recorded in a saved execution artifact. Metrics and interpretations follow established guidance that chatbot evaluation should separate component accuracy from overall conversational quality and should account for dataset artifacts and measurement bias [24]–[27].

A. Evaluation Metrics (Accuracy, Precision, Recall, F1-score)

Two metric groups are reported:

- **Intent classification (component-level):** Accuracy and macro-averaged Precision/Recall/F1 across the closed intent set.
- **End-to-end system behavior:** pass rate on scenario tests (functional acceptance), out-of-scope (OOS) detection count, and response latency summary (min/median/avg/max).

Macro-averaged scores are emphasized because the intent set is multi-class and exhibits class imbalance; macro scores reduce the risk of majority-class dominance masking poor performance on smaller intents.

B. Experimental Setup

Intent classification evaluation. The evaluation uses the repository intent patterns in `data/intents.json`, which contains 650 preprocessed patterns across 18 intent labels. A stratified 80/20 split is applied (521 train, 129 test) with a fixed random seed (42). The same preprocessing logic used by the implementation (`text_preprocessor.py`) is applied prior to training and evaluation.

Baselines are derived from components available in the repository:

- **Majority baseline:** always predicts the most frequent training intent.
- **Keyword rules:** assigns the intent with the highest keyword match count (using the `keywords` fields in the intent JSON).
- **TF–IDF (1-NN cosine):** nearest-neighbor matching in TF–IDF space.
- **TF–IDF + Logistic Regression:** multi-class linear classifier over TF–IDF uni/bi-grams.
- **Sentence-Transformers semantic similarity:** `all-MiniLM-L6-v2` cosine similarity against training patterns, matching the repository's semantic intent logic.
- **Weighted ensemble:** semantic (0.75) + TF–IDF (0.25) intent selection using the decision rules implemented in `intent_model.py`.

End-to-end system tests. End-to-end results are taken from `TEST_RESULTS_20260405_153750.txt`, which executes 35 scenario tests spanning in-domain requests, hybrid queries, explicit OOS queries, and boundary inputs.

C. Performance Results

1) Intent classification (holdout split on repository patterns): **Observations.** Keyword rules improve substantially over the majority baseline, indicating that intent-specific lexical cues exist in the pattern set. Among fast lexical baselines, TF–IDF nearest-neighbor provides the strongest macro F1, while logistic regression provides slightly better macro precision. The semantic component provides the strongest performance overall (accuracy 0.783; macro F1 0.756). In this split, the weighted ensemble matches the semantic predictor because the semantic signal dominates under the default weights; the

TABLE II
INTENT CLASSIFICATION PERFORMANCE (MACRO-AVERAGED).

Model	Acc.	P	R	F1
Majority baseline	0.146	0.008	0.056	0.014
Keyword rules	0.620	0.646	0.539	0.557
TF–IDF (1-NN cosine)	0.620	0.634	0.539	0.562
TF–IDF + Logistic Regression	0.612	0.652	0.478	0.505
Sentence-Transformers (MiniLM-L6-v2)	**0.783**	**0.773**	**0.771**	**0.756**
Weighted ensemble (0.75 sem + 0.25 TF–IDF)	**0.783**	**0.773**	**0.771**	**0.756**

TABLE III
END-TO-END AUTOMATED TEST OUTCOMES FROM `TEST_RESULTS_20260405_153750.TXT`.

Metric	Value
Total tests	35
Passed	33 (94.3%)
Failed	2 (5.7%)
OOS detected	6
Latency (median / avg / max)	2046.8 / 4370.6 / 13011.8 ms

ensemble's primary benefit is calibration and robustness in edge cases where lexical evidence is decisive. These numbers are best interpreted as pattern-set generalization rather than a full measure of real-user robustness; pattern corpora can contain artifacts that inflate apparent performance [25], [26].

2) End-to-end automated test outcomes (saved execution artifact): Category-level pass counts in the same artifact indicate that failures are isolated to boundary inputs (whitespace-only and punctuation-only queries), while in-domain and explicit OOS categories pass.

D. Latency Breakdown (NLU vs LLM)

The end-to-end artifact reports multi-second response latencies but does not separate local NLU computation from LLM response time. The application implementation exposes an LLM timing signal (`response_time`), enabling a decomposition of per-turn latency into local NLU, LLM inference, and fixed overhead. A local microbenchmark (CPU, after warm-up) indicates that the control layer is comparatively fast: intent-only inference has a median of 27.97 ms (mean 28.55 ms), and intent+emotion+scope has a median of 60.62 ms (mean 63.67 ms; $p95$ 83.86 ms). Consequently, the multi-second tail observed in the end-to-end artifact is primarily attributable to external LLM inference and network variability rather than local intent routing.

E. Statistical Strength of Scenario Tests

The recorded end-to-end scenario result (33/35 passed) corresponds to a pass rate of 94.3%. However, the sample size is small; a 95% Wilson confidence interval is approximately [81.4%, 98.4%], indicating substantial uncertainty. To reduce this uncertainty, the evaluation should be expanded to at least 100–150 scenarios covering paraphrases, borderline OOS queries, hybrid queries, and noisy inputs, consistent with recommendations to interpret chatbot scores conservatively under dataset and measurement artifacts [25]–[27].

F. Comparison with Baseline Methods (if available)

The component-level comparison suggests that rule-based routing can be competitive in precision but may be limited in recall, reflecting reduced coverage for paraphrases and short queries. TF–IDF baselines improve macro recall and macro F1, supporting the use of statistical models for intent routing.

The repository implementation additionally includes a semantic similarity component (Sentence-Transformers) and a weighted ensemble. On the same holdout split, the semantic component substantially improves macro F1 (0.756), consistent with prior work emphasizing robust representations for intent routing and uncertainty-aware scoping [17], [18].

G. Error Analysis

Intent misclassification patterns (TF–IDF + Logistic Regression). On the 129-example holdout test split, the TF–IDF + logistic regression baseline produces 50 misclassifications. Representative errors reveal systematic ambiguity and overlap between intents:

- **Semantic overlap:** "hostel charges included?" is predicted as `hostel` instead of `fees`, reflecting that cost-related queries can cross multiple intents.
- **Short conversational acts:** greetings/thanks/negation (e.g., "hello there", "thank you so much", "not at all") are misrouted, suggesting insufficient conversational-act coverage or overly aggressive stop-word removal.
- **Slot-driven queries:** "how do i find my seat number?" is predicted as `admission` instead of `exams`, indicating that some exam-administration terms are not strongly represented in the learned lexical features.

End-to-end failure modes. The only recorded failures occur on boundary inputs (whitespace-only and punctuation-only). Such degenerate inputs are known to trigger brittle behavior in NLU pipelines and motivate explicit normalization and routing rules [28], [29].

Strengths and weaknesses. Strength is a strong end-to-end functional pass rate on in-domain and OOS scenarios in the recorded artifact, indicating stable orchestration and conservative fallback behavior. Weakness is intent confusion for conversational-act intents and semantically overlapping fee/hostel/admission categories; improving label definitions, increasing pattern coverage, and adding explicit conversational-act handling are expected to improve macro recall.

V. CONCLUSION

This work addresses the problem of delivering a reliable college-domain assistant that can route user queries to appropriate intents, handle out-of-scope (OOS) inputs conservatively, and produce helpful responses while reducing brittle behavior in real-world interaction.

Approach. The system is implemented as a controlled conversational pipeline that combines preprocessing, ensemble intent inference, scope enforcement, emotion and tone cues, and an LLM response layer with knowledge grounding and operational safeguards (e.g., fallback and recovery). The design reflects evidence that robust deployed assistants require both strong representations and explicit control mechanisms, particularly for OOS handling and noise resilience [17], [18], [28].

Key findings. Quantitative results on repository artifacts indicate: (i) lexical baselines achieve up to 0.620 accuracy and up to 0.562 macro F1 on the held-out split of intent patterns, while the semantic component achieves 0.783 accuracy and 0.756 macro F1 on the same split; and (ii) end-to-end scenario tests achieve a 94.3% pass rate (33/35; 95% CI approximately [81.4%, 98.4%]), with failures concentrated on degenerate boundary inputs and a median latency of 2046.8 ms in the saved test artifact.

Contributions.

- A modular, repository-grounded architecture for a college assistant integrating intent routing, context management, scope gating, and an LLM layer.
- A practical evaluation setup that reports component-level intent metrics and end-to-end scenario outcomes from saved execution artifacts.
- Operational safeguards (confidence-aware routing, conservative fallback behavior, and error recovery) aligned with deployment reliability goals.

Limitations.

- Intent evaluation is based on locally authored patterns rather than large-scale external benchmarks, and may overestimate generalization due to dataset artifacts [25], [26].
- The intent evaluation reflects pattern-set generalization and does not fully capture real-user distribution shift, multi-turn ambiguity, or adversarial/noisy inputs.
- Boundary input handling (e.g., punctuation-only and whitespace-only) remains a failure point and motivates stronger normalization and routing rules.

Future work.

- Multilingual support: extend preprocessing and intent representations to multilingual and code-mixed inputs.
- Better context handling: strengthen dialogue state tracking and long-horizon memory to reduce ambiguity in multi-turn queries.
- Real-time deployment: optimize latency and concurrency, and validate behavior under load with production-oriented monitoring.
- Improved robustness: expand adversarial/noise testing, harden normalization, and improve OOS calibration under distribution shift.

VI. References

The bibliography for this report is defined in `references.bib`.

References

[1] S. Hussain, O. A. Sianaki, and N. Ababneh, "A survey on conversational agents/chatbots classification and design techniques," in *Web, Artificial Intelligence and Network Applications: Proceedings of the Workshops of the 33rd International Conference on Advanced Information Networking and Applications (WAINA-2019)*. Springer, 2019, pp. 946–956.

[2] G. Caldarini, S. Jaf, and K. McGarry, "A literature survey of recent advances in chatbots," *Information*, vol. 13, no. 1, p. 41, 2022.

[3] B. Liu and I. Lane, "Joint intent classification and slot filling with recurrent neural networks," in *Proceedings of Interspeech*, 2016.

[4] D. Hakkani-Tur *et al.*, "Multi-domain joint semantic frame parsing using bi-directional rnn-lstm," in *Proceedings of Interspeech*, 2016.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019.

[7] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[8] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, Tech. Rep., 2018.

[9] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.

[10] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, 2020.

[11] C. Khatri *et al.*, "Advancing the state of the art in open domain dialog systems through the alexa prize," in *Proceedings of Alexa Prize Workshop*, 2018.

[12] S. Roller *et al.*, "Recipes for building an open-domain chatbot," in *Proceedings of ACL*, 2021.

[13] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, 2022.

[14] R. Thoppilan *et al.*, "Lamda: Language models for dialog applications," *arXiv preprint arXiv:2201.08239*, 2022.

[15] OpenAI, "Chatgpt: Optimizing language models for dialogue," https://openai.com/blog/chatgpt, 2022.

[16] J. Gao, M. Galley, and L. Li, "Neural approaches to conversational ai," *Foundations and Trends in Information Retrieval*, 2019.

[17] S. Larson, A. Mahendran, J. Peper, C. Clarke, A. Lee, P. Hill, J. K. Kummerfeld, K. Leach, M. Laurenzano, L. Tang, and J. Mars, "An evaluation dataset for intent classification and out-of-scope prediction," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 1311–1316.

[18] I. Casanueva, T. Temcinas, D. Gerz, M. Henderson, and I. Vulic, "Efficient intent detection with dual sentence encoders," in *Proceedings of the 2nd Workshop on NLP for Conversational AI*, 2020, pp. 38–45.

[19] Q. Chen, Z. Zhuo, and W. Wang, "Bert for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019.

[20] C. Xia, C. Zhang, X. Yan, Y. Chang, and P. Yu, "Capsule neural networks for joint slot filling and intent detection," in *Proceedings of ACL*, 2018.

[21] C. Zhang *et al.*, "Few-shot intent detection via contrastive pretraining and fine-tuning," in *Proceedings of EMNLP*, 2020.

[22] S. Kim *et al.*, "Dialogue state tracking: A neural reading comprehension approach," in *Proceedings of SIGDial*, 2019.

[23] C.-W. Huang *et al.*, "Transferable multi-domain state generator for task-oriented dialogue systems," in *Proceedings of ACL*, 2020.

[24] W. Maroengsit, T. Piyakulpinyo, K. Phonyiam *et al.*, "A survey on evaluation methods for chatbots," in *Proceedings of the 2019 7th International Conference on Information and Education Technology*. ACM, 2019.

[25] S. Gururangan, S. Swayamdipta, O. Levy, R. Schwartz, S. R. Bowman, and N. A. Smith, "Annotation artifacts in natural language inference data," in *Proceedings of NAACL-HLT*, 2018, pp. 107–112.

[26] M. Geva, Y. Goldberg, and J. Berant, "Are we modeling the task or the annotator? an investigation of annotator bias in natural language understanding datasets," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 1161–1166.

[27] S. Swayamdipta, R. Schwartz, N. Lourie, Y. Wang, H. Hajishirzi, N. A. Smith, and Y. Choi, "Dataset cartography: Mapping and diagnosing datasets with training dynamics," in *Proceedings of EMNLP*, 2020, pp. 9275–9293.

[28] Y. Belinkov and Y. Bisk, "Synthetic and natural noise both break neural machine translation," in *Proceedings of ICLR*, 2018.

[29] Z. Sun, A. Li, and X. Qiu, "Adversarial training for code-mixed language understanding," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1364–1373.

[30] J. Gao, M. Galley, and L. Li, "Recent advances in conversational ai," *AI Magazine*, 2020.

[31] T.-H. Wen *et al.*, "A network-based end-to-end trainable task-oriented dialogue system," in *Proceedings of EACL*, 2017.

[32] A. Bordes, Y.-L. Boureau, and J. Weston, "Learning end-to-end goal-oriented dialog," in *Proceedings of ICLR*, 2017.